Appendix A 解答

GA-Q01

1. Blocks：

$$\lceil 1{,}000{,}000/256 \rceil = \lceil 3906.25 \rceil = 3907.$$

2. 前 3906 blocks 各 256 threads，共

$$3906 \times 256 = 999{,}936.$$

最後 block active threads：

$$1{,}000{,}000 - 999{,}936 = 64.$$

剩 $256 - 64 = 192$ threads 要因 `global_id >= N` 而不執行 memory access。

3. 第一個 element：`blockIdx.x=0, threadIdx.x=0, global_id=0`。

最後 element：`blockIdx.x=3906`。Block base 是 $3906 \times 256 = 999{,}936$，last element index 是 999,999，所以：

$$threadIdx.x = 64 - 1 = 63, \qquad global_id = 999999.$$

4. shared-memory資源上限是$\lfloor 48/12 \rfloor = 4$ blocks。題目明定其他資源不受限，因此本題最多4 blocks。不能借用另一題的768-thread上限改答3；若指定該上限，才需再取 $\min(4, \lfloor 768/256 \rfloor) = 3$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

GA-Q02

1. 兩條 mutually exclusive paths 各執行一個 masked pass，共 2 passes。
2. 兩個 passes 各有 32 issued lane slots，useful lane-thread executions是 $5 + 27 = 32$：

$$32/(2 \times 32) = 50\%.$$

3. 若只有 if 且 path 長度一個 instruction block，useful 5，issued lane slots 32：

$$5/32 = 15.625\%.$$

4. Predication 會讓全部 lanes 執行 predicated instructions，只是 inactive lanes 不 commit result；它避免 branch/reconvergence overhead，卻仍消耗issue slots；predicate為false的lane不應真的執行其memory load/store，不能把inactive lane算成必然的DRAM transaction。若 body 長或只有少數 lanes active，branching 可能較省。

GA-Q03

1. 32 lanes × 4 bytes =128 contiguous bytes，每 sector 32 bytes：

$$128/32 = 4 \text{ sectors}.$$

Appendix A 解答 （續）

2. Stride 8 words 是 $8 \times 4 = 32$ bytes，所以每個 lane 落在不同 sector，共 32 sectors。Useful bytes 是 128，sector bytes 是 $32 \times 32 = 1024$：

$$128/1024 = 12.5\%.$$

3. 連續 128 bytes 從 sector 內 +4 bytes 開始，會跨 5 個 32-byte segments：第一個與最後一個 partially used。實際 transaction 數仍取決於 sector/line granularity 與 hardware request formation。
4. Sector 被搬動不代表所有 requested words 位於同一 DRAM row/bank/page；row activation、bank conflicts、channel balance、cache misses、outstanding requests 與 L2 filtering 也影響 achieved bandwidth。Transaction count 只是 locality 的一部分。

GA-Q04

1. 沒有 conflict。Lanes 的 word addresses consecutive，mod 32 分別對應 32 個 banks。
2. 所有 addresses 的 bank 相同，但 addresses 不同，形成 32-way bank conflict，serialize 成多個 service groups。
3. 同一地址 read 可由 hardware broadcast/multicast 一次服務，通常不算 bank conflict。只有在同一 bank 的 different addresses 才造成 conflict。
4. 常用解法是 padding：把 row stride 從 32 words 改為 33 words，使 threads' bank indices 分散。仍非保證最佳，因為 access pattern、shared-memory footprint、sync overhead、不同 instruction phases 都可能改變 conflict；需用 profiler 實測。

GA-Q05

1. Kernel A，每 block 256 threads：

- threads：$768/256 = 3$ blocks。
- blocks：hardware 8，不是限制。
- registers：每 block $256 \times 32 = 8192$，$32768/8192 = 4$。
- shared memory：$48/16 = 3$。

Static minimum 是 3 blocks。

2. Active threads $3 \times 256 = 768$。Theoretical occupancy：

$$768/768 = 100\%,$$

共 24 warps。

3. 128-thread block、每 thread 32 registers、每 block 16 KiB：

Appendix A 解答 （續）

- threads：6 blocks。
- blocks：8。
- registers：每 block 4096，允許 8 blocks。
- shared：3 blocks。

Minimum 3 blocks，active threads 384，occupancy $384/768 = 50\%$。Shared memory 成為限制。若 shared 隨 block 縮成 8 KiB，shared 允許 6 blocks，則 threads 與 shared 都是 6-block limit，active threads 768。

4. 50% occupancy 的 kernel 可能有更多 register ILP、較少 shared-memory latency、更少 cache thrash、較低 divergence 或更高 instruction-level parallelism；若 50% warps 已足以覆蓋 latency，增加 threads 只會增加 contention。要看 stall reason 與 achieved throughput。

GA-Q06

1. Rows：

- Row 0：`[Ap[0],Ap[1]) = [0,2)`，entries `(0,3)`、`(2,-1)`。
- Row 1：`[2,3)`，entry `(1,4)`。
- Row 2：`[3,5)`，entries `(0,2)`、`(2,5)`。

2. Multiply-add：

$$y[0] = 3(2) + (-1)(3) = 3.$$
$$y[1] = 4(10) = 40.$$
$$y[2] = 2(2) + 5(3) = 19.$$

所以 $y = [3, 40, 19]$。

3. Row work 分別 2、1、2 nonzero。在warp-per-row中，這三rows各只有2、1、2個nonzeros，一輪分工的有用lane比例至多為2/32、1/32、2/32；row長度差還會造成不同warps完成時間不均。若改為thread-per-row，才是同一warp中不同row的threads因迭代長度不同而產生masked-loop浪費。應考慮 warp-per-row、block-per-row、動態分配或 reorder rows。

4. `x[Aj[k]]` indirect addresses 常重複使用但 scattered，shared/read-only cache 可降低 global reads。若 working set 太大、conflicts 多、cache load/barrier overhead 高，或 `x` 很小且已 L1/L2 hit，搬入 shared memory 不一定划算。

GA-Q07

1.

Appendix A 解答 （續）

$$I_{\text{ridge}} = P_C/B = 250/320 = 0.78125 \text{ FLOP/byte}.$$

Single-level roof：

$$P \le \min(P_C, IB).$$

2. Kernel A：

$$IB = 0.3(320) = 96 \text{ GFLOP/s},$$
$$P_A \le \min(250, 96) = 96 \text{ GFLOP/s},$$

memory-bound。

3. Kernel B：

$$IB = 4(320) = 1280 \text{ GFLOP/s}, \qquad P_B \le \min(250, 1280) = 250 \text{ GFLOP/s},$$

compute-bound。

4.

$$P_{\text{measured}} = 1.2 \times 10^9/0.005 = 2.4 \times 10^{11} \text{ FLOP/s} = 240 \text{ GFLOP/s}.$$

240 超過 AI=0.3、B=320 所給的 96 GFLOP/s memory bound，因此這組輸入不可能描述同一個完整 phase。必須檢查 FLOP 是否少算或時間是否只含部分 kernel、intensity 是否用錯資料層的 bytes、cache 命中是否讓實際 DRAM 流量低於假設，以及 GB 與 GiB 或 ms 與 s 的單位。不能為超過上界的測量硬加 divergence 或 occupancy 解釋，因為那些成本只會降低效能。

5.

$$I_D = 10^9/(0.25 \times 10^9) = 4 \text{ FLOP/byte},$$
$$P_D \le \min(1000, 4(100)) = 400 \text{ GFLOP/s}.$$
$$I_S = 10^9/(4 \times 10^9) = 0.25 \text{ FLOP/byte},$$
$$P_S \le \min(1000, 0.25(1000)) = 250 \text{ GFLOP/s}.$$
$$P \le \min(1000, 400, 250) = 250 \text{ GFLOP/s}.$$
$$T_D = 0.25 \times 10^9/(100 \times 10^9) = 2.5 \text{ ms},$$
$$T_S = 4 \times 10^9/(1000 \times 10^9) = 4 \text{ ms},$$
$$T \ge \max(1, 2.5, 4) = 4 \text{ ms}.$$

$(4, 400)$ 是 DRAM 層斜線上的頻寬上界；$(0.25, 250)$ 是 shared 層斜線上的值。兩層合併後的有效上界是 250 GFLOP/s；shared roof 的 plateau 仍為 1000，不是 250。

Appendix A 解答 （續）

GA-Q08

1. Useful=8×2+24×6=160，issued=32×(2+6)=256，utilization=160/256=62.5%。分子按active lanes乘該path長度，不是只把32個threads各算一次。
2. 50%簡式只在兩個互斥paths等長且兩者皆被執行時成立。若B完全無active lane而A有32個，省略B，只issue A的2個instructions，利用率64/64=100%；不能仍把LB加進分母。
3. 不可以。還缺指令本身的latency/throughput、memory transactions與等待、同步、launch/host-device copy、CPU基準等。Lane utilization是同一warp的執行slot指標，不是跨平台elapsed-time比值。

GA-Q09

1. Addresses0,8,...,248，sector=floor(8i/32)=floor(i/4)，共有0–7八個sectors。每sector4個有用words，共128 bytes，占8×32=256 bytes的50%。
2. 28+4i涵蓋bytes28–155，sectors0–4，共5個；128/160=80%。Stride128時每lane不同sector，共32，128/1024=12.5%。這些數字只計一次warp request的sector traffic，不保證下一層DRAM傳輸數相同。
3. Bank=34i mod32=2i mod32。最小重複週期16，故16個banks各有兩個不同words，2-way conflict。35i mod32=3i mod32，gcd(3,32)=1，32個i對應32個不同banks。一般L的週期32/gcd(L,32)，完整32-lane request每bank重複gcd(L,32)次，前提是words不同且本模型不另有swizzle/多port。
4. Shared配置以block為整數單位，padding可能使resident blocks由3降2；增加barrier成本、較差其他phase layout或降低occupancy，都可抵銷單次column read的改善。需在相同workload下比較完整執行時間，不能只比較衝突倍數。

Appendix A 解答 （續）

GA-Q10

步驟	array
初始補0	[2,0,1,3,4,0,0,0]
up d=1	[2,2,1,4,4,4,0,0]
up d=2	[2,2,1,6,4,4,0,4]
up d=4	[2,2,1,6,4,4,0,10]
root=0	[2,2,1,6,4,4,0,0]
down d=4	[2,2,1,0,4,4,0,6]
down d=2	[2,0,1,2,4,6,0,10]
down d=1	[0,2,2,3,6,10,10,10]

前5項為[0,2,2,3,6]；總和從up root另存為10。Up與down各7次加法，共14，抽象tree depth各3，共6輪，另計root初始化/同步常數。Padding維持數學和不變，輸出只取原本N=5個位置，不能把identity產生的延伸prefix誤當多出的原資料。

Down-sweep的parent prefix p是其整個區間之前的和，left subtree總和t由up-sweep已正確算得。Left區間之前仍是p；right區間之前多了全部left，故為p+t。Root之前沒有元素，p=0，所以按樹高歸納所有leaf i取得sum(a[0:i])。這個證明適用任意合法整數輸入，不只本表。

多block的local scan每一塊都從0起算，漏掉前面blocks的總和。必須scan block sums取得offset，再加回各block。若直接拼接，例如第二塊第一個輸出會是0，但全域prefix應是第一塊總和。

GA-Q11

1. z=[0,0,0,1,0,0,1,1]，exclusive Z_i=[0,0,0,0,1,1,1,2]，總Z=3。Dest zero=Z_i，one=3+i−Z_i，得到[3,4,5,0,6,7,1,2]。
2. 三輪如下。

完成bit	(key,identity)順序
0	[(0,d),(2,g),(6,h),(5,a),(1,b),(5,c),(1,e),(7,f)]
1	[(0,d),(5,a),(1,b),(5,c),(1,e),(2,g),(6,h),(7,f)]
2	[(0,d),(1,b),(1,e),(2,g),(5,a),(5,c),(6,h),(7,f)]

Appendix A 解答 （續）

3. 第j個zero前面恰有j個zeros，故dest依序0..Z−1；第j個one前面恰有j個ones，dest依序Z..N−1。兩區不重疊且合起來覆蓋N個位置，沒有scatter write race。每區目的位置按原順序增加，所以stable。假設先按低bit得到[0,1]，再按相同高bit把兩者反序成[1,0]，就破壞已排序的低bit；歸納需要同高bit區內保留前輪順序。重複keys的a/c與b/e順序在最後仍保留。

GA-Q12

1. 距離2。a0=Gm1/2^2=3/4，a1=−Gm0/2^2=−1/2；F0=m0a0=3/2、F1=m1a1=−3/2，總force0。兩個mass不同時acceleration大小不必相等，a0+a1=1/4並不違反動量守恆；應核對mass-weighted總和。
2. Naive sources=N^2=64，tiled有N/B=2個blocks，每block讀8個sources，所以16；另加8個my loads，總72與24，bytes分別1152與384。Source traffic降4倍，但總traffic降3倍，不可把所有層級的bytes都除以4。此模型忽略cache及compiler的其他重用。
3. 最後block的i=8..11只有前兩個target有效，但四個threads仍參與每輪載入與barrier。Source tile最後base=8時，threads0、1載入body8、9，threads2、3放identity/zero body；計算只遍歷min(B,N−base)=2個有效sources。若無效target提早return，可能使有效threads等待未到達barrier的participants，或缺少它們負責載入的source。正確程式用predicate保護運算而非跳過共同同步。
4. T_GPU(K)=16.777216+2.02K ms，T_CPU=25K。要求25K/(16.777216+2.02K)>=5，等價於14.9K>=83.88608，K>=5.62994，因此最小整數6。K=6時speedup=150/28.897216約5.1908；K=5時125/26.877216約4.6508，仍未達5。此推導明定不能overlap且中間資料留device，若每輪都copy便需重建模型。

Appendix A 解答 （續）

原創驗算程式附錄: gpu_models.py

```
"""Small reproducible models for SIMT divergence, sectors, CSR SpMV and roofline."""

from dataclasses import dataclass


def grid_blocks(elements: int, threads_per_block: int) -> int:
    return (elements + threads_per_block - 1) // threads_per_block


def unique_sectors(words: list[int], sector_bytes: int = 32) -> int:
    return len({(address * 4) // sector_bytes for address in words})


def csr_spmv(ap: list[int], aj: list[int], av: list[int], x: list[float]) -> list[float]:
    if len(ap) < 1:
        raise ValueError("Ap must contain at least one sentinel")
    if ap[0] != 0 or ap[-1] != len(av) or len(aj) != len(av):
        raise ValueError("invalid CSR arrays")
    output = []
    for row in range(len(ap) - 1):
        start, stop = ap[row], ap[row + 1]
        output.append(sum(av[k] * x[aj[k]] for k in range(start, stop)))
    return output


def occupancy_blocks(
    threads_per_block: int,
    registers_per_thread: int,
    shared_kib_per_block: int,
    *,
    sm_threads: int = 768,
    sm_blocks: int = 8,
    sm_registers: int = 32768,
    sm_shared_kib: int = 48,
) -> dict[str, int]:
    return {
        "threads": sm_threads // threads_per_block,
        "blocks": sm_blocks,
        "registers": sm_registers // (threads_per_block * registers_per_thread),
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Appendix A 解答 （續）

```
        "shared": sm_shared_kib // shared_kib_per_block,
    }


def main() -> None:
    assert grid_blocks(1_000_000, 256) == 3907
    contiguous = unique_sectors(list(range(32)))
    strided = unique_sectors([8 * lane for lane in range(32)])
    misaligned = unique_sectors([1 + lane for lane in range(32)])
    assert (contiguous, strided, misaligned) == (4, 32, 5)

    y = csr_spmv([0, 2, 3, 5], [0, 2, 1, 0, 2], [3, -1, 4, 2, 5], [2, 10, 3])
    assert y == [3, 40, 19]

    a = occupancy_blocks(256, 32, 16)
    b = occupancy_blocks(128, 32, 16)
    assert min(a.values()) == 3
    assert min(b.values()) == 3
    assert 3 * 256 / 768 == 1.0
    assert 3 * 128 / 768 == 0.5

    memory_roof_a = 320 * 0.3
    bound_a = min(250, memory_roof_a)
    bound_b = min(250, 320 * 4)
    assert (memory_roof_a, bound_a, bound_b) == (96, 96, 250)

    print(f"grid blocks: {grid_blocks(1_000_000, 256)}")
    print(f"sectors contiguous/strided/misaligned: {contiguous}/{strided}/{misaligned}")
    print(f"CSR y: {y}")
    print(f"occupancy A limits: {a}; B limits: {b}")
    print(f"roofline: AI=0.3 -> {bound_a} GFLOP/s; AI=4 -> {bound_b} GFLOP/s")
    print("GPU model checks: PASS")


if __name__ == "__main__":
    main()
```

Appendix A 解答 （續）

原創驗算程式附錄: revision_models.py

```
"""Cost, hierarchy, stack, and coherence models used by the revised manuscripts."""

from dataclasses import dataclass


@dataclass(frozen=True)
class OperationClass:
    count: int
    baseline_cycles_each: float
    local_speedup: float


def cost_breakdown(classes: tuple[OperationClass, ...]) -> tuple[list[float], list[float]]:
    before = [item.count * item.baseline_cycles_each for item in classes]
    after = [time / item.local_speedup for time, item in zip(before, classes)]
    return before, after


@dataclass(frozen=True)
class MemoryBoundary:
    name: str
    bytes_transferred: float
    bytes_per_second: float


def hierarchical_bound(flops: float, compute_flops_per_second: float, boundaries:
tuple[MemoryBoundary, ...]) -> dict:
    levels = [{"name": level.name, "bytes": level.bytes_transferred,
               "intensity": flops / level.bytes_transferred,
               "bound_flops_per_second": flops / level.bytes_transferred *
level.bytes_per_second,
               "minimum_seconds": level.bytes_transferred / level.bytes_per_second}
              for level in boundaries]
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Appendix A 解答 （續）

```
    compute_seconds = flops / compute_flops_per_second
    return {"levels": levels, "bound_flops_per_second": min(compute_flops_per_second, *
(level["bound_flops_per_second"] for level in levels)),
            "minimum_seconds": max(compute_seconds, *(level["minimum_seconds"] for level in
levels))}


def p1_cases() -> dict:
    mixed = cost_breakdown((OperationClass(80, 1, 8), OperationClass(20, 10, 1)))
    equal = cost_breakdown((OperationClass(80, 1, 8), OperationClass(20, 1, 1)))
    base = hierarchical_bound(1e9, 1e12, (MemoryBoundary("DRAM", 4e9, 100e9),
MemoryBoundary("shared", 4e9, 1000e9)))
    tiled = hierarchical_bound(1e9, 1e12, (MemoryBoundary("DRAM", .25e9, 100e9),
MemoryBoundary("shared", 4e9, 1000e9)))
    return {"mixed_cost": {"before": mixed[0], "after": mixed[1], "speedup":
sum(mixed[0])/sum(mixed[1])},
            "equal_cost": {"before": equal[0], "after": equal[1], "speedup":
sum(equal[0])/sum(equal[1])},
            "untiled": base, "tiled": tiled,
            "wrong_reused_dram_intensity_bound": min(1e12, 4 * 1000e9)}


if __name__ == "__main__":
    import json
    result = p1_cases()
    assert result["mixed_cost"]["before"] == [80, 200]
    assert result["mixed_cost"]["after"] == [10, 200]
    assert abs(result["mixed_cost"]["speedup"] - 4/3) < 1e-12
    assert abs(result["equal_cost"]["speedup"] - 10/3) < 1e-12
    assert result["untiled"]["bound_flops_per_second"] == 25e9
    assert result["tiled"]["bound_flops_per_second"] == 250e9
    assert result["tiled"]["minimum_seconds"] == .004
    assert result["wrong_reused_dram_intensity_bound"] == 1e12
    print(json.dumps(result, indent=2))
    print("revision cost and hierarchical traffic models: PASS")
```

參考資料與署名

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study